RDBMS vs GDBMS: A Performance Benchmark

Data Management

Luca Di Carlo Gabriele Bruni

Sapienza University of Rome
July Xth, 2026

Presentation Overview

Project Overview

The Challenge

Relational Databases (RDBMS) struggle to scale when handling highly interconnected data. Queries requiring multiple ”hops” translate into massive, expensive JOINs.

Our Proposed Solution

- **Comparative Analysis:** A direct benchmark between PostgreSQL (RDBMS) and Neo4j (GDBMS).
- **Data Domain:** Academic co-authorship network based on a subset of the arXiv dataset.
- **Goal:** Demonstrate in which scenarios the Property Graph model outperforms the tabular model, evaluating both raw performance and ease of development.

Architectural Differences

- **PostgreSQL (Set-based):** Relationships are computed at runtime via Foreign Keys and JOIN operations. Cost tends to scale with query depth.
- **Neo4j (Path-based):** Relationships are stored as physical pointers (Index-Free Adjacency). Traversals are localized to the immediate neighbourhood of the nodes.

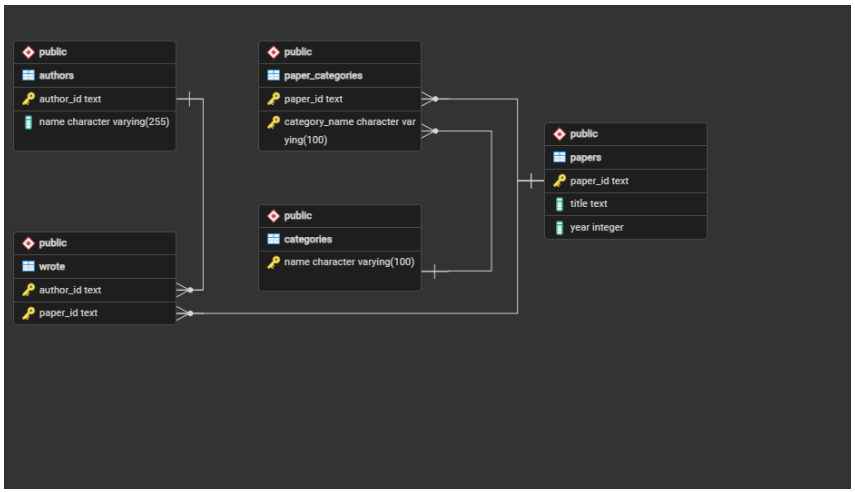


Figure: ER Schema (RDBMS)

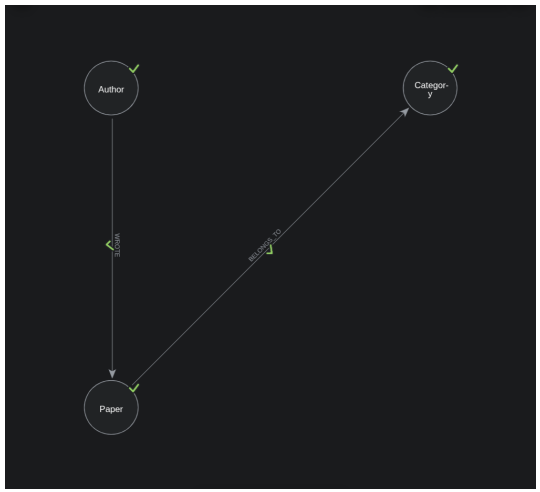


Figure: Property Graph (GDBMS)

Problem Statement

The Core Question

When should we abandon the reliability of an RDBMS in favor of a GDBMS?

PostgreSQL

- *Structure:* De facto standard for Relational Databases.
- *Strengths:* Excellent for global aggregations (`GROUP BY`).
- *Language (SQL):* Declarative, math-based. Verbose for deep relations.

Neo4j

- *Structure:* Graph Database leader.
- *Strengths:* Optimized for pattern matching and shortest paths.
- *Language (Cypher):* Visual pattern-matching. Highly intuitive.

Dataset & Logical Schema

arXiv Scientific Papers Dataset

- **Authors:** Researchers identified by unique IDs.
- **Papers:** Academic publications (Title, Year) identified by unique IDs.
- **Categories:** Scientific domains (e.g., cs.AI, cs.DB).

Relationship	Source Node	Target Node
`WROTE`	Author	Paper
`BELONGS_TO`	Paper	Category

Table: Relationship Mapping

ETL Pipeline

From Source to Database

- **RDBMS Setup:** every step has to be written by hand: DDL for the schema, Python scripts to clean and reshape the data, and manual enforcement of Foreign Key constraints. Loading the CSVs also gave us some friction directly (encoding/path issues when importing from the repository).
- **GDBMS Setup:** Neo4j's import tool is interactive: you map nodes and relationships visually in the UI, and it generates the *Cypher* import script for you. No bridge tables to design.

- **Extraction:** Python script parsing the raw JSON into CSVs.
- **Indexing Strategy:** Neo4j is indexed on primary identifiers automatically when you declare a Node Key Constraint during import, while in PostgreSQL every index has to be created explicitly (B^+-Tree).

- **Query 1: Aggregation**
  Find the 15 most prolific authors in a decade. A classic task where SQL is historically strong.

- **Query 2: Deep Traversal**
  Find 2^{nd}-degree co-authors of a specific target. A stress test for SQL's multiple `JOIN`s vs Cypher's graph traversal.

- **Query 3: Shortest Path**
  Calculate the degrees of separation between two authors. Native graph algorithm vs Relational recursive queries.

Results & Analysis

Execution Times

Query Type	PostgreSQL	Neo4j	Winner
1. Aggregation	523 ms	**416 ms**	Neo4j
2. Deep Traversal	**835 ms**	58590 ms	PostgreSQL
3. Shortest Path	OoT*	**340 ms**	Neo4j

*OoT = Out of Time (Query was manually terminated after 60 minutes)
Note: Times measure engine execution, excluding network latency or GUI.

Query 1: Aggregation

Most Prolific Authors (2014-2023)

Task

Count the publications of each author and rank the top 15.

- **Performance:** Both engines are practically on par and exceptionally fast. While SQL is highly optimized for aggregations, Neo4j slightly edges out PostgreSQL here, proving GDBMS can handle global tabulations efficiently.
- **Ease of Writing:** SQL is mathematically built for this kind of task but Cypher also feels like an intuitive approach.
- **Our Insight:** In Cypher, the use of a `MATCH` clause with a pattern saves us the burden of a triple SQL `JOIN`.

Query 1: The Code

PostgreSQL (SQL)

```
SELECT a.name AS "Author",
       COUNT(DISTINCT p.paper_id) AS "
            NumberOfPublications"
FROM authors a
JOIN wrote w ON a.author_id = w.author_id
JOIN papers p ON w.paper_id = p.paper_id
JOIN paper_categories pc ON p.paper_id = pc.
     paper_id
WHERE p.year >= 2014 AND p.year <= 2023
GROUP BY a.name
ORDER BY "NumberOfPublications" DESC
LIMIT 15;
```

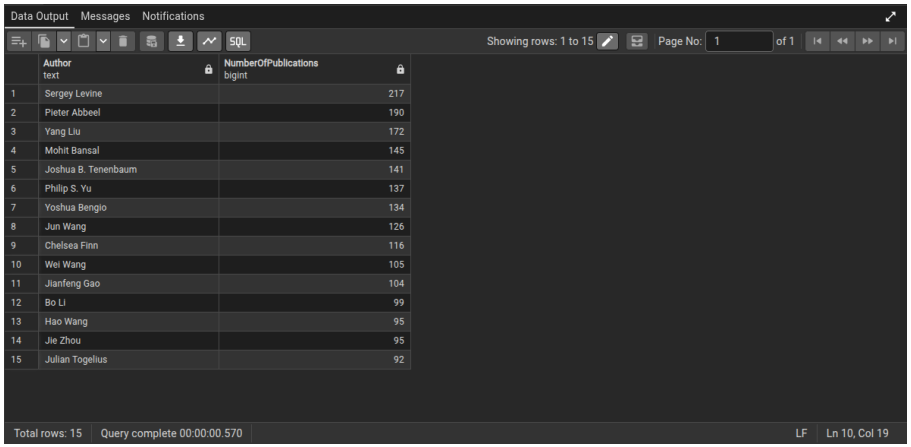
Neo4j (Cypher)

```
MATCH (a:Author)-[:WROTE]->(p:Paper)-[:
     BELONGS_TO]->(c:Category)
WHERE p.year >= 2014 AND p.year <= 2023
RETURN a.name AS Author,
       COUNT(DISTINCT p) AS
            NumberOfPublications
ORDER BY NumberOfPublications DESC
LIMIT 15;
```

Query 1: The Results

PostgreSQL

Without Indexes (Unfair): ***570 ms***



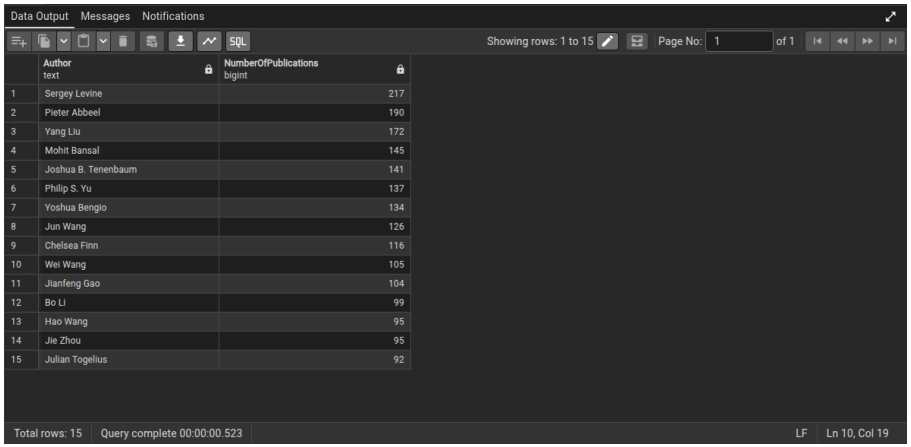
Data Output | Messages | Notifications

Showing rows: 1 to 15 | Page No: 1 of 1

	Author (text)	NumberOfPublications (bigint)
1	Sergey Levine	217
2	Pieter Abbeel	190
3	Yang Liu	172
4	Mohit Bansal	145
5	Joshua B. Tenenbaum	141
6	Philip S. Yu	137
7	Yoshua Bengio	134
8	Jun Wang	126
9	Chelsea Finn	116
10	Wei Wang	105
11	Jianfeng Gao	104
12	Bo Li	99
13	Hao Wang	95
14	Jie Zhou	95
15	Julian Togelius	92

Total rows: 15 | Query complete 00:00:00.570 | LF | Ln 10, Col 19

With Indexes (Fair): ***523 ms***



Data Output | Messages | Notifications

Showing rows: 1 to 15 | Page No: 1 of 1

	Author (text)	NumberOfPublications (bigint)
1	Sergey Levine	217
2	Pieter Abbeel	190
3	Yang Liu	172
4	Mohit Bansal	145
5	Joshua B. Tenenbaum	141
6	Philip S. Yu	137
7	Yoshua Bengio	134
8	Jun Wang	126
9	Chelsea Finn	116
10	Wei Wang	105
11	Jianfeng Gao	104
12	Bo Li	99
13	Hao Wang	95
14	Jie Zhou	95
15	Julian Togelius	92

Total rows: 15 | Query complete 00:00:00.523 | LF | Ln 10, Col 19

Neo4j

Cypher Execution: ***416 ms***

Author	NumberOfPublications
"Sergey Levine"	217
"Pieter Abbeel"	190
"Yang Liu"	172
"Mohit Bansal"	145
"Joshua B. Tenenbaum"	141
"Philip S. Yu"	137
"Yoshua Bengio"	134
"Jun Wang"	126
"Chelsea Finn"	116
"Wei Wang"	105
"Jianfeng Gao"	104
"Bo Li"	99
"Hao Wang"	95
"Jie Zhou"	95
"Julian Togelius"	92

Started streaming 15 records after 2 ms and completed after 416 ms.

Query 2: Deep Traversal

The "Super-Node" Anomaly

Task

Find the top 15 2^{nd}-degree co-authors for a specific target (e.g., Sergey Levine), excluding direct 1^{st}-degree collaborators.

- **SQL (Set-based):** PostgreSQL computes the 1^{st}-degree "blacklist" **only once** in memory using CTEs, applying it via highly efficient *Hash Joins*.
- **Cypher (Path-based):** Neo4j recalculates the `NOT` exclusion **on every single path**. Because the target is a *Super-Node* (thousands of links), this triggers a massive combinatorial explosion.
- **Our Insight:** Cypher is elegant but hides costs on Super-Nodes. It requires manual optimization (e.g., using `collect()`) to force the set-based behavior SQL applies natively.

Query 2: The Code

PostgreSQL (SQL)

```
WITH Target AS (
  SELECT author_id FROM authors
  WHERE name = 'Sergey Levine'
),
DirCo AS (
  SELECT DISTINCT w2.author_id FROM Target t
  JOIN wrote w1 ON t.author_id = w1.author_id
  JOIN wrote w2 ON w1.paper_id = w2.paper_id
  WHERE w2.author_id != t.author_id
)
SELECT a2.name AS "2ndDegree",
       COUNT(w4.paper_id) AS "Collabs"
FROM Target t
JOIN wrote w1 ON t.author_id = w1.author_id
JOIN wrote w2 ON w1.paper_id = w2.paper_id
JOIN wrote w3 ON w2.author_id = w3.author_id
JOIN wrote w4 ON w3.paper_id = w4.paper_id
JOIN authors a2 ON w4.author_id = a2.
     author_id
WHERE w2.author_id != t.author_id
  AND w2.paper_id != w3.paper_id
  AND w3.author_id != w4.author_id
  AND w4.author_id != t.author_id
  AND w4.author_id NOT IN (SELECT author_id
                           FROM DirCo)
GROUP BY a2.name
ORDER BY "Collabs" DESC LIMIT 15;
```

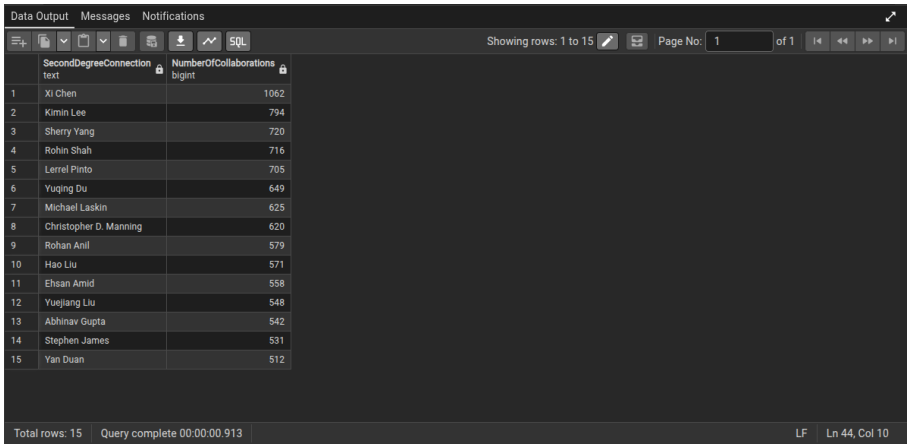
Neo4j (Cypher)

```
MATCH (target:Author {name: 'Sergey Levine'})
      -[:WROTE]->(p1:Paper)
      <-[:WROTE]-(coauthor:Author)
      -[:WROTE]->(p2:Paper)
      <-[:WROTE]-(co_coauthor:Author)
WHERE target <> co_coauthor
  AND NOT (target)-[:WROTE]->()
          <-[:WROTE]-(co_coauthor)
RETURN co_coauthor.name AS SecondDegree,
       count(p2) AS Collabs
ORDER BY Collabs DESC LIMIT 15;
```

Query 2: The Results

PostgreSQL

Without Indexes (Unfair): ***913 ms***



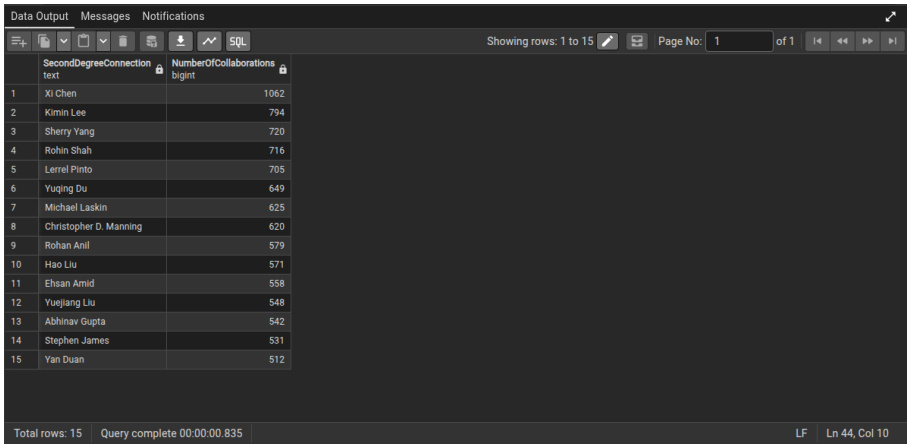
Data Output | Messages | Notifications

Showing rows: 1 to 15 | Page No: 1 of 1

	SecondDegreeConnection text	NumberOfCollaborations bigint
1	Xi Chen	1062
2	Kimin Lee	794
3	Sherry Yang	720
4	Rohin Shah	716
5	Lerrel Pinto	705
6	Yuqing Du	649
7	Michael Laskin	625
8	Christopher D. Manning	620
9	Rohan Anil	579
10	Hao Liu	571
11	Ehsan Amid	558
12	Yuejiang Liu	548
13	Abhinav Gupta	542
14	Stephen James	531
15	Yan Duan	512

Total rows: 15 | Query complete 00:00:00.913 | LF | Ln 44, Col 10

With Indexes (Fair): ***835 ms***



Data Output | Messages | Notifications

Showing rows: 1 to 15 | Page No: 1 of 1

	SecondDegreeConnection text	NumberOfCollaborations bigint
1	Xi Chen	1062
2	Kimin Lee	794
3	Sherry Yang	720
4	Rohin Shah	716
5	Lerrel Pinto	705
6	Yuqing Du	649
7	Michael Laskin	625
8	Christopher D. Manning	620
9	Rohan Anil	579
10	Hao Liu	571
11	Ehsan Amid	558
12	Yuejiang Liu	548
13	Abhinav Gupta	542
14	Stephen James	531
15	Yan Duan	512

Total rows: 15 | Query complete 00:00:00.835 | LF | Ln 44, Col 10

Neo4j

Cypher Execution: ***58590 ms***

SecondDegreeConnection	NumberOfCollaborations
"Xi Chen"	1062
"Kimin Lee"	794
"Sherry Yang"	720
"Rohin Shah"	716
"Lerrel Pinto"	705
"Yuqing Du"	649
"Michael Laskin"	625
"Christopher D. Manning"	620
"Rohan Anil"	579
"Hao Liu"	571
"Ehsan Amid"	558
"Yuejiang Liu"	548
"Abhinav Gupta"	542
"Stephen James"	531
"Yan Duan"	512

Started streaming 15 records after 479 ms and completed after 58,590 ms.

Query 3: Shortest Path

Degrees of Separation

Task

Find the minimum number of hops connecting two specific authors.

- **Performance:** GDBMS drastically outperforms RDBMS. Neo4j excels here using a bi-directional BFS algorithm that terminates immediately when the path is found.
- **RDBMS Complexity:** PostgreSQL uses a recursive CTE to perform a search, which consumes significant memory as the path depth increases.
- **Our Insight:** Writing a recursive CTE in SQL is complex (15+ lines of code). Cypher solves it intuitively:
  `shortestPath((a)-[*..10]-(b))`.

Query 3: The Code

PostgreSQL (SQL)

```
WITH RECURSIVE bfs(curr, len, visited) AS (
  SELECT author_id, 0, ARRAY[author_id]
  FROM authors WHERE name = 'Sergey Levine'

  UNION ALL

  SELECT w2.author_id, b.len + 1,
         b.visited || w2.author_id
  FROM bfs b
  JOIN wrote w1 ON b.curr = w1.author_id
  JOIN wrote w2 ON w1.paper_id = w2.paper_id
  WHERE b.len < 5
    AND w2.author_id != ALL(b.visited)
)
SELECT b.visited AS "PathArray",
       b.len AS "SeparationDegrees"
FROM bfs b JOIN authors a ON b.curr = a.
      author_id
WHERE a.name = 'Xi Chen'
ORDER BY b.len ASC LIMIT 1;
```

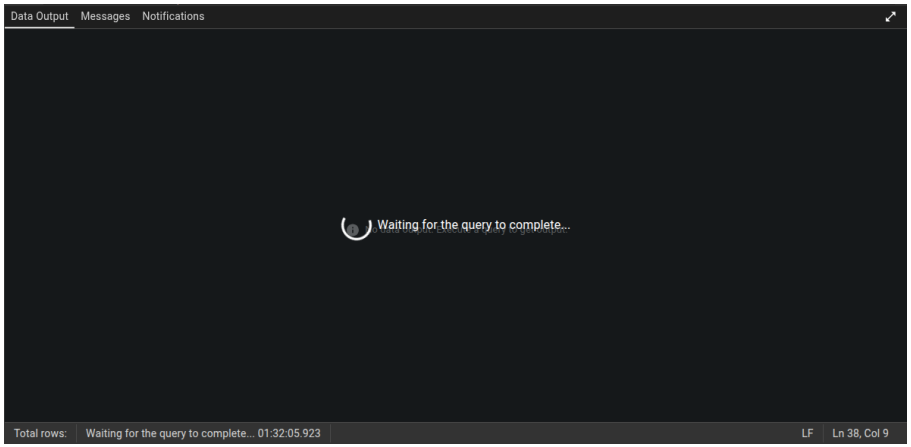
Neo4j (Cypher)

```
MATCH (start:Author {name: 'Sergey Levine'})
MATCH (end:Author {name: 'Xi Chen'})
MATCH path = shortestPath(
      (start)-[:WROTE*..10]-(end)
)
RETURN path, length(path) / 2 AS
      SeparationDegrees;
```

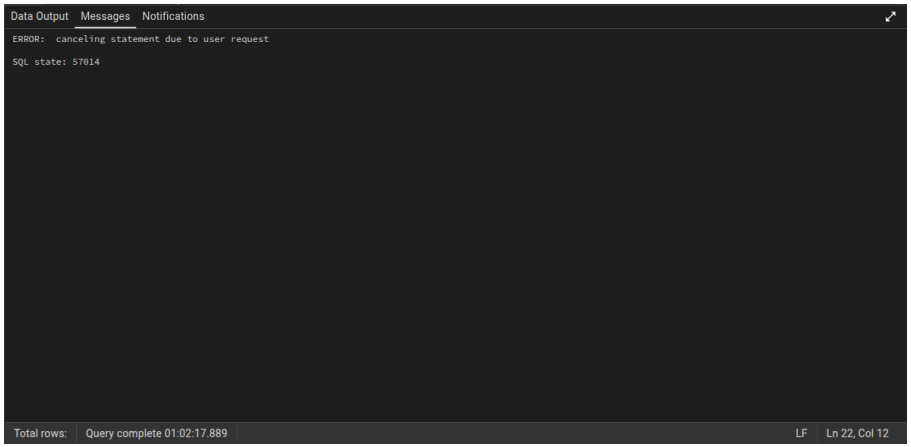
Query 3: The Results

PostgreSQL

Without Indexes (Unfair): ***Out of Time***

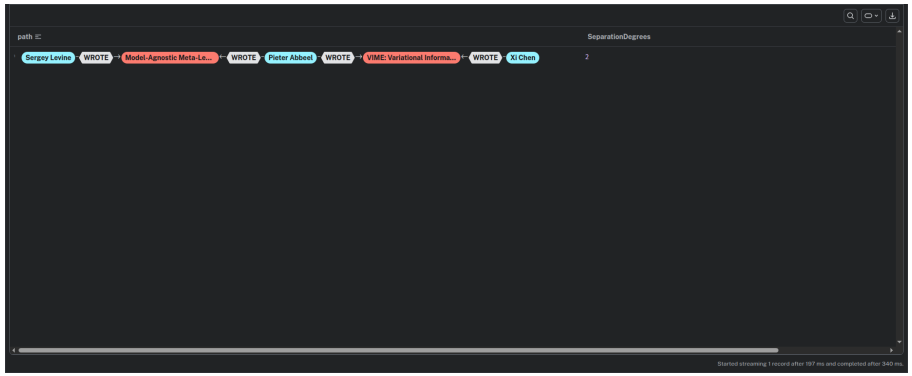


With Indexes (Fair): ***Out of Time***



Neo4j

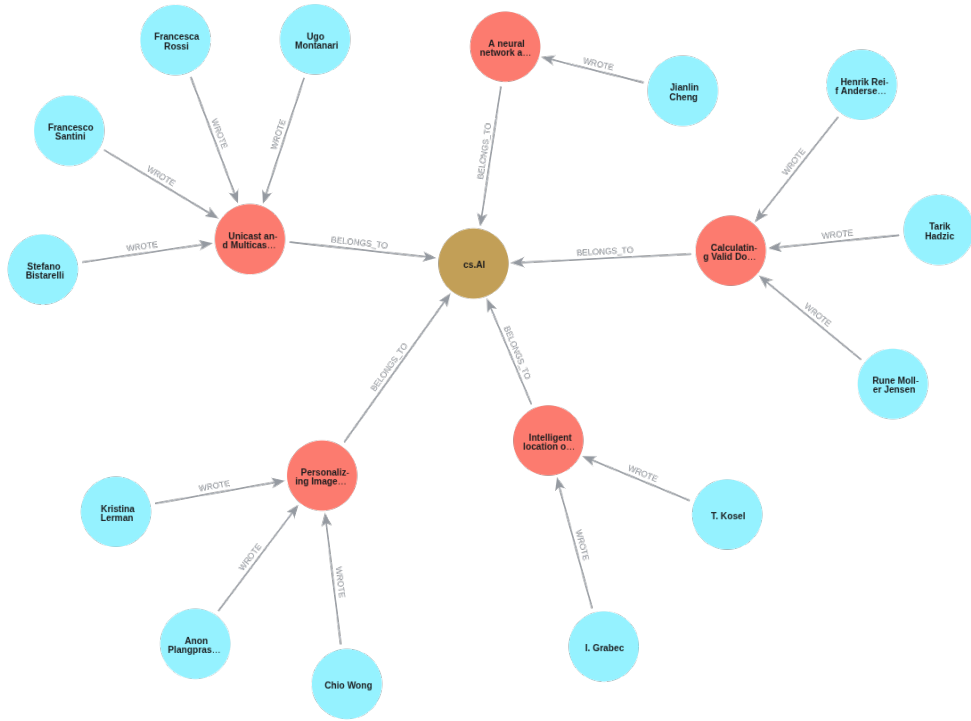
Cypher Execution: ***340 ms***



Conclusions

- **RDBMS Strengths:** Unbeatable for structured aggregations and situations where the query optimizer leverages set-based math (Hash Joins).
- **GDBMS Strengths:** Undoubtedly the winner for recursive queries, shortest paths, and deeply connected data where JOINs collapse.
- **Our Experience:** Neo4j simplifies the ETL pipeline and makes querying complex patterns visually intuitive.
- **Final Verdict:** Graph Databases do not replace Relational Databases. They are the correct tool to adopt alongside an RDBMS when the value shifts from the ”data” to the ”relationships”.

Thank you for your attention!



Any Questions?

Luca Di Carlo Gabriele Bruni